

Real-time Assistive Reader Pen for Arabic Language

MOHAMMAD A. ALZUBAIDI, MWAFFAQ OTOOM, and NOURAN S. AHMAD,
Yarmouk University

Disability is an impairment affecting an individual's livelihood and independence. Assistive technology enables the disabled cohort of the community to break the barriers to learning, access information, contribute to the community, and live independently. This article proposes an assistive device to enable people with visual disabilities and learning disabilities to access printed Arabic material in real-time, and to help them participate in the education system and the professional workforce.

This proposed assistive device employs Optical Character Recognition (OCR) and Text To Speech (TTS) conversion, using concatenation synthesis. OCR is achieved using image processing, character extraction, and classification, while Arabic speech synthesis is achieved through concatenation synthesis, followed by Multi Band Re-synthesis Overlap-Add (MBROLA). Waveform generation in the second phase produces vocal output for the disabled user to hear. OCR character and word accuracy tests were conducted for nine Arabic fonts. The results show that six fonts were recognized with over 60% character accuracy and two fonts were recognized with over 88% accuracy. A Mean Opinion Score (MOS) test for speech quality was conducted. The results showed an overall MOS score of 3.53/5 and indicated that users were able to understand the speech. A real-time usability testing was conducted with 10 subjects. The results showed an overall average of agreements scores of 3.9/5 and indicated that the proposed Arabic reader pen meets the real-time constraints and is pleasant and satisfying to use and can contribute to make printed Arabic material accessible to visually impaired persons and people with learning disabilities.

CCS Concepts: • **Human-centered computing** → **Accessibility**; **Accessibility technologies**; • **Computer systems organization** → **Embedded and cyber-physical systems**; **Embedded systems**; **Embedded software**;

Additional Key Words and Phrases: Arabic language, assistive embedded systems, optical character recognition, reader pen, real-time systems, text-to-speech

ACM Reference format:
Mohammad A. Alzubaidi, Mwaffaq Otoom, and Nouran S. Ahmad. 2021. Real-time Assistive Reader Pen for Arabic Language. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* 20, 1, Article 13 (March 2021), 30 pages.
https://doi.org/10.1145/3423133

1 INTRODUCTION

Disability is an impairment, a complex phenomenon reflecting the interaction between a person and the society in which he or she lives. It can be congenital, or it might be acquired at any stage of an individual's lifetime. It can substantially affect a person's life activities. Impairments

Authors' addresses: M. A. Alzubaidi, M. Otoom, and N. S. Ahmad, Computer Engineering Department, Yarmouk University, Irbid, 21163, Jordan; emails: {maalzubaidi, mof.otoom}@yu.edu.jo, nearat.nouran@gmail.com.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2021 Association for Computing Machinery.
2375-4699/2021/03-ART13 $15.00
https://doi.org/10.1145/3423133

are classified as either cognitive, developmental, intellectual, mental, physical, sensory, or a combination of them [1].

The World Health Organization (WHO) estimates that more than a billion people have a disability of some form, which corresponds to about 15% of the global population [2].

Many students with disabilities are unable to attend public schools and are only allowed to attend special-needs schools, where they can receive special education. This keeps them isolated from the society [4]. However, there are 420M people around the world who speak Arabic, and more than 1.5B Muslims who might need to read Arabic.

Assistive technology aims to make the mainstream educational system more accessible to the disabled and enable them to contribute to the community. It also increases their independence by allowing them to carry out tasks that they would otherwise be unable to perform without assistance [5].

A reader pen is a Text-To-Speech assistive device that alleviates learning difficulties associated with visual impairment and learning disabilities by translating input text into speech waveforms. The aim of a universal TTS program is to transform all types of textual input that a human being is able to pronounce, regardless of its complexity [6]. One example of a reader pen is C-Pen, which is designed to scan printed text and then play it as audible speech for the user to hear. It is currently available in languages such as English and Spanish [7].

The current challenges in building an assistive reader device are (1) to meet the necessary real-time constraints and (2) to deliver natural, human-like, accurate speech—especially for less supported languages, such as Arabic. In doing so, certain special requirements for Arabic must be met, such as supporting diacritics and providing correct pronunciation of letters and abbreviations. Ideally, such a device would be implemented within the limited resources available in portable embedded systems and would deliver high-quality Arabic speech to people with visual impairments or learning difficulties.

Optical Character Recognition (OCR) and speech synthesis are the two primary focuses of the research in this article. We employ a traditional approach to OCR (using character extraction and classification) and a novel approach to speech synthesis, which is accomplished through diphone concatenation synthesis, and the MBROLA algorithm.

Previous studies (which will be described in the next section) have concentrated on speech synthesis for languages such as English, French, and Spanish. Only a few speech synthesis frameworks support Arabic, and most of them are implemented to run on a personal computer or a mobile device. These platforms provide much greater resources than a compact embedded device, which has much less memory and more constrained processing power.

The contribution of this research is toward the development of a real-time TTS reader pen for Arabic, implemented using a System on a Chip (SoC). Such a compact device is an essential tool for the visually impaired to access textual content that they encounter moment-by-moment in their daily life.

This article is organized as follows: In Section 2, we explore the related work. The methodology is explained in the next three consecutive sections, where Section 3 describes the Reader System Algorithms, Section 4 describes the hardware implementation, and Section 5 describes the experimental setup. In Section 6, we discuss the results of the conducted experiments. We conclude the article in Section 7.

2 LITERATURE REVIEW

In this section, we review the state-of-the-art work on Text-To-Speech (TTS) and Optical Character Recognition (OCR) technologies, including those that support Arabic language. In addition, we review similar assistive reading pen devices that are presented in the literature.

2.1 Text-To-Speech (TTS)

2.1.1 Concatenation Synthesis. One popular TTS method is concatenation synthesis. This method strings together previously recorded words and syllables to generate a near-human-like speech stream. However, speech streams generated using this method have noticeable glitches. Therefore, the quality of the generated speech is not as natural as desired to the human ear [8].

Sanaki [9] introduced an embedded real-time TTS apparatus. In this research, an Advanced RISC Machine receives a stream of input text from a keyboard. This text stream is routed to a PIC microcontroller, which is connected to a speakJet IC. The speakJet IC converts the text stream into a speech stream, using concatenation synthesis, and routes the speech stream to a loudspeaker to output the audio. The downside to this approach is that it requires a lot of hardware, which can be costly and bulky. Moreover, the speakJet IC only supports English.

Hamad and Hussain [10] developed a TTS system for Arabic that relies on concatenation synthesis of allophones and diphones. In this research, the software was developed using the Microsoft Speech API, which requires the Microsoft Windows operating system. The system receives text in a phonetic format and generates speech in Arabic with an accuracy of around 75%.

Praveen Kumar Bamini [11] developed an FPGA implementation of a concatenation speech synthesis algorithm. It employed FPGA flip-flops to provide a relatively small in-memory database of audio recordings, called an acoustic library. Each recording represents a word segment consisting of four letters or less. The speech synthesis algorithm receives segmented words and searches the library for a matching phone for each segment. The concatenation of the audio recordings is done using MATLAB.

The study of Tora et al. [12] demonstrates a hardware-based Turkish TTS implementation, using an external voice synthesizer card. The TTS system is built using an RC8660 voice synthesizer chip, which supports English by default. However, it can be configured to generate speech for other languages by adding a phoneme mapping dictionary. The Mean Opinion Score (MOS) of the system is 3.29 out of 5. However, the RC8660 voice synthesizer chip is limited to languages with ASCII characters. Therefore, Arabic is not supported [13].

Chabchoub and Cherif [14] presented an implementation using Time Domain Pitch Synchronous Overlap-Add (TD-PSOLA) tools to enhance the quality of an Arabic TTS system, which was based on Diphone concatenation. The resulting speech was tested by a group of listeners using two tests: the Diagnostic Rhyme Test (DRT) and the Categorical Estimation (CE) test. The test scores showed an enhancement in accuracy and naturalness. However, the TD-PSOLA algorithm is considered computationally intensive.

2.1.2 Unit Selection Synthesis. Unit selection synthesis is another speech synthesis technology that relies on a database of speech recordings. These recordings are segmented before database creation into a large number of waveform combinations. Accordingly, this method stores phones and diphones, plus samples of half-phones, morphemes, syllables, words, phrases, and even full sentences [15].

Bachan and Tokarski [16] researched the creation and evaluation of synthesized speech for Polish using MaryTTS, which is an open source Java framework for speech synthesis. The chosen method for speech synthesis was unit-selection, which is available in MaryTTS. The authors use Python and Praat scripts for text manipulations. According to this research, the accuracy of the speech synthesis for Polish was over 80%, which is relatively high. However, the MarryTTS framework does not yet support Arabic.

Sarathy and Ramakrishnan [17] researched the development of Text-To-Speech for an embedded mobile device. The synthesis method used is unit selection concatenative–based, where a speech unit is selected from the database based on its phonetic context. This article also studies the tradeoff

between database size and the quality of synthetic speech in the mobile environment. Several speech compression schemes used in mobile devices are tested and compared, according to their quality. The paper proposes a methodology that uses the existing speech codec in mobile devices to read SMS text messages to the listener.

The research of Vanitha et al. [18] demonstrates real-time TTS hardware, based on a Raspberry Pi. The Clustergen algorithm is the speech synthesis method used in this implementation. It employs a parametric synthesis cluster-based unit selection method. The software in the system is programmed using the C language. It accepts text input from a keyboard that is connected to the Raspberry Pi. and converts that text to corresponding speech. However, the quality of the generated speech was not addressed by the authors.

The study of Rebai and BenAyed [19] presents an efficient Arabic TTS system with a diacritization engine that uses unit selection for speech synthesis. In this work, the authors proposed an implementation using deep neural networks to predict the correct Arabic diacritics. Experimental results indicate high-precision diacritic generation and high-quality synthesized speech with an MOS score between 3.42 and 3.67 out of 5.

In summary, unit selection is capable of achieving the greatest level of output naturalness by relying on a large database. However, such large databases are not practical for embedded, resource-constrained systems.

2.1.3 Neural Network–based Synthesis. Frontini and Malcangi [20] reported a neural network–based method for implementing a TTS system and compared it to a rule-based system. The neural network is trained using the back-propagation algorithm with an adaptive learning rate. The authors developed a software tool called NetTalk using MATLAB and Java programming.

Lakshmi et al. [21] implemented TTS conversion using a Raspberry Pi. Their system combines both optical character recognition and TTS synthesis to create a device that scans English text from an image and converts it to its corresponding speech audio. The hardware used is Raspberry Pi 2, along with a Raspberry Pi Camera Module. The software used for OCR is Tesseract, which is an open source OCR engine. For the TTS, the authors used the Google Speech API, which uses neural networks for speech synthesis.

Fogarassy-Neszly and Pribeanu [22] proposed a software module for TTS synthesis supporting multiple languages. In this article, language detection algorithms are used and tested on texts of different languages. The software used for speech synthesis in this article is the Microsoft Speech API (SAPI). The authors developed multiple versions of their software. The commercial version has been used in a screen reader application and in a personal planner application for the visually impaired. The research includes usability and acceptance tests of the applications with visually impaired and blind users.

Tacotron [23], a software engine built using Python language, relies on a single neural network trained from only data to produce linguistic and acoustic features. Tacotron uses the Griffin-Lim algorithm for phase estimation to vocode the magnitude spectrograms. Initially Tacotron only supported English language. Later in 2019, a multilingual update was published [24], but Tacotron users will need to create training data for the chosen language.

2.1.4 Formant Synthesis. Formant Synthesis is a speech synthesis method that does not use pre-recorded human speech samples to create a synthesized output. Instead, it uses acoustic modeling and additive synthesis, which takes a stream of fundamental frequency and noise levels as input parameters to generate artificial speech [25].

Lukose and Upadhya [26] presented a Speech Synthesizer based on Formant Synthesis. The synthesis method used is Cascade Formant Synthesis and is implemented for synthesizing English vowels. Although the formant synthesis method produces robotic and unnatural sound, this

approach showed high accuracy evaluation results for their implementation. That evaluation was done using Mean Opinion Score (MOS), which is based on how well the listener can clearly understand the output speech.

The artificial voice generated using formant synthesis method is robotic-sounding, which is quite distinguishable from natural speech. However, in systems where a high level of naturalness is not the main goal, formant synthesis can be useful for its reliability and for reducing anomalies in the output when compared to concatenative-based systems. Formant synthesis programs also need less memory than concatenation-based programs, since they do not rely on a large data-store of pre-recorded audio. Accordingly, this method is preferable for embedded devices with limited storage and processing power [6].

2.1.5 TTS for Arabic. Zerrouki et al. [50] structured new voices and defined new TTS conversion rules to mainly tackle diacritization in Arabic for integration with eSpeak. Zangar et al. [51] studied the class-specific modeling approach of the duration in the Arabic sounds to be used for text-to-speech synthesis, and compared it with other modeling approaches such as the HMM-based and the DNN-based modeling. Zine and Meziane [52] proposed a sub-segments-based approach for concatenative Arabic TTS. The authors measured the intelligibility of the synthesized speech using the Diagnostic Rhyme Test (DRT). In a later work, Zine et al. [53] studied the improvement of the Arabic TTS using a sub-segments-based approach by adopting a lemma-based approach for concatenative Arabic TTS.

Amrouche et al. [54] presented a design and implementation of Diacritic Arabic TTS, called GArabic TTS, under MATLAB environment. Abdelali et al. [55] presented automatic diacritization of Maghrebi Arabic dialects, Tunisian and Moroccan, using a character-level deep neural network architecture. Abed et al. [56] studied the effect of diacritics on Arabic speech recognition systems by building and comparing diacritized and nondiacritized models for different corpus sizes. Darwish et al. [57] presented a new Arabic diacritizer that predicts the diacritics of words and then their case endings. A Viterbi decoder was used for diacritics of words and SVM for case endings.

Abdelmalek and Mnasri [58] modified the original unit selection algorithm to be used to design a high-quality Arabic concatenative speech synthesis system, by integrating more phonological, linguistic, and contextual features. Alsaif et al. [59] developed an intelligent Arabic TTS mobile application. They used a set of statistical language models, n-gram for word prediction, and auto-completion for easy typing. For that, a large new Arabic corpus for daily communication in different domains was constructed.

Abdo et al. [60] built a modern standard Arabic audio-visual corpus for TTS. Hadj Ali et al. [61] presented gemination prediction in an Arabic TTS automatic diacritization module using DNNs. Zine et al. [62] developed a web-based assistive platform, based on a TTS API, and used it for learning Arabic language. Rebai and BenAyed [63] presented an Arabic TTS system based on a statistical parametric approach and non-uniform units speech synthesis. It includes a diacritization engine. Fahmy et al. [64] presented an Arabic speech synthesizer using an end-to-end neural deep network architecture called Tacotron 2. It uses a pre-trained English model and a total of 2.41 hours of recorded speech, followed by Wave-Glow to synthesize Arabic speech. Elharati et al. [65] established an automatic Arabic speech recognition system using MATLAB based on MFCC and discrete-observation multivariate HMM and 24 CVCVCV Arabic words.

2.2 Optical Character Recognition (OCR) for Arabic

There are a few works on Arabic document layout analysis, such as the work of Bouressace and Csirik [66], in which they applied a convolutional neural network (CNN) to segment an Arabic document image, captured by a smartphone, into its elements. These elements are categorized

into 10 classes, including text, author, figure, table, and so on. Eltay et al. [67] explored various deep learning architectures to recognize handwritten Arabic texts offline. That work focused on the issue of having imbalanced datasets for deep learning. Arora et al. [68] presented a text recognition system based on open vocabulary sub-words using the hybrid deep neural network HMM (DNN-HMM) acoustic model used in the automatic speech recognition (ASR). Mohamad et al. [69] recognized printed Arabic letters using their different shapes within words by defining new geometrical features. Mohammad et al. [70] developed a robust offline character segmentation algorithm for printed Arabic text with diacritics based on a contour extraction technique.

Mustafa and Elbashir [71] presented a deep-learning-based method to recognize common Arabic names very quickly, without any segmentation. Qaroush et al. [72] investigated word and character segmentation for printed Arabic text. Word segmentation was performed using a vertical projection and Interquartile Range (IQR) method to differentiate between word gaps and within-word gaps. For character segmentation, a projection profile method was used, along with a set of statistical and topological features that are font-independent. Al-Sheikh et al. [73] reviewed an Arabic OCR dataset of handwritten, printed, and embedded texts. This dataset can be used in different learning algorithms for recognition. The authors also reviewed existing work on Arabic text recognition using various deep learning approaches. Milo and Martínez [74] proposed a new OCR strategy for Arabic using typographic technology based on deep analysis of Arabic script structures.

Darwish and Elzoghaly [75] presented a new offline Arabic OCR model for printed text based on a Genetic Algorithm (GA) and the Fuzzy K-Nearest Neighbor classifier (F-KNN) to enhance the identification accuracy. GA was used as a feature selection algorithm. The F-KNN algorithm was used to classify uncertain data objects. Kadi and Nasri [76] presented a method to extract the features of isolated Arabic characters based on the Hough Transform. Hussein and Hussain [77] proposed a hybrid of a decision tree and an Artificial Neural Network (ANN) for an online Arabic handwriting isolated character recognition system, through the identification of its features and dots.

2.3 Similar Assistive Reading Pen Devices

Ok and Rao [78] demonstrated how digital pens can be used to support learning of students with learning disabilities. The authors surveyed existing digital pens such as Reading pens, Optical Character Recognition (OCR) pens, Anybook Readers, and Smart pens. WizCom Tech [79] has developed a reading pen that can be used to scan printed words or lines, record them, and read them. This reader pen also includes a translation function. Examples of OCR pens include C-Pen [80], IRISPen [81], and PenPower WorldPenScan [82]. These are mainly portable scanners the size of a pen. Some of them are Bluetooth-enabled, facilitating the transfer of the recognized text to mobiles or computers, in an editable format. The Anybook readers [78] allow teachers to attach audio notes on the text. Students can then access these audio notes to better understand the text. Smart pens such as Livescribe pens (and its predecessor the FLY Fusion Pen) [83] can be used for taking digital notes from handwritten text, including recording audio and synchronizing it with the handwritten text.

Huang et al. [84] introduced a digital pen named Glissade. Glissade applied haptic feedback on digital pens, called balance shifting feedback, to enhance interactions. There are many studies investigated the benefits of using digital pens in learning. For instance, Chen et al. [85] used the Livescribe smart pen to support paper-based classroom English learning. Chen et al. [86] used the Que-voice digital pen to facilitate oral reading fluency for English learners. The Que-voice contains a built-in speaker and microphone and can read printed text. Tan et al. [87] evaluated the effectiveness of digital pen–based learning for improving the listening comprehension of English

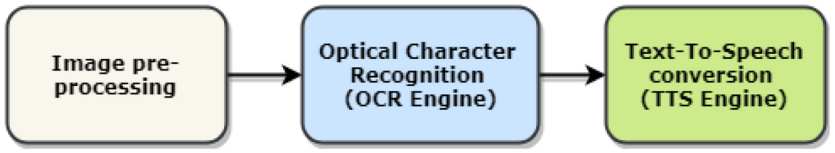


Fig. 1. The flowchart of the reader pen algorithm.

learners. Choi et al. [88] evaluated the use digital pens during English picture book reading for increasing a child's interest in learning English. Krish [89] investigated the effectiveness of using audio pen technology to assist in English reading fluency and comprehension and vocabulary skills among preschool children.

2.4 Summary

Multiple speech synthesis technologies have been used to generate artificial speech signals, with each technology having its pros and cons, depending on the field of application. The majority of these studies have concentrated on speech synthesis for high-level applications. A few have been implemented using embedded devices. However, those do not support Arabic. Researchers have also been working on several approaches to implement real-time reading systems. Among all the related works mentioned above, there has been little work in real-time reading embedded devices for Arabic, which requires complex processing and more variations in language rules and pronunciation. Some of those embedded systems require phonemes as input, so text-to-phoneme conversion must be done prior to the TTS conversion. Others require manual configuration of a database of recordings. Those that are fully automated do not support Arabic.

Several works have been done on Arabic OCR using different image processing, object recognition, and machine learning approaches. However, none of them has been designed to work with embedded systems in real-time.

Last, a number of similar reading pen assistive devices have been introduced in the literature. Unfortunately, none of them support the use of Arabic language. Moreover, the majority of them are commercial and thus not open source.

With this in mind, this article proposes to answer the following research question:

How could we build a real-time assistive embedded system that serves as a Text-To-Speech reader pen for the Arabic language?

3 READER SYSTEM ALGORITHM

In this section, we describe the algorithms that we used for image preprocessing, OCR and speech synthesis. A set of three main processes are executed to get the desired result, as shown in the flowchart in Figure 1. Basic image pre-processing steps are done to reduce the noise of the captured images, convert them to binary images, and normalize and standardize them. Optical Character Recognition (OCR) is then used to convert the pre-processed images to text. Last, Text-To-Speech (TTS) is used to convert the resulting text into speech. We used existing open source tools: Tesseract [29] for OCR and eSpeak NG [46] for TTS, with some tweaking and configurations, as will be explained in the following subsections. The developed programs, tools, and resources will be available (upon request) as open source for developers and researchers.

3.1 Image Preprocessing

The purpose of this step is to prepare a stream of images of text captured by a camera to be processed by the OCR engine [28].

There are three pre-processing steps for text images:

3.1.1 Noise Reduction. Text images can be noisy. Visual noise is random variation of pixel intensities within an image. Noise reduction is used to remove light and dark spots from the image, making the text easier to classify by an OCR engine in the next step.

There are many filters that could be used for reducing noise. The choice of filter depends on the type of noise within the image. Most digital images have Gaussian noise. To filter Gaussian noise, we use a median filter. This also removes other types of noise.

3.1.2 Grayscale to Binary Conversion Using Adaptive Thresholding. The conventional thresholding operation employs a global fixed threshold for all pixels across the image. Adaptive thresholding (which iteratively calculates a local threshold across the image) involves more computation than global thresholding. However, it produces better results with images that have variations in illumination or shadows.

One approach to adaptive thresholding examines the intensity values of the local neighborhood around each pixel and computes the mean value as a sum of the neighboring pixel intensities divided by the neighborhood block size, as shown in Equation (1):

$$\text{Threshold} = \frac{\sum_{i=0}^{N} pixel_intensity(i)}{\text{neighbor_block_size}} \tag{1}$$

The size of the neighborhood must be large enough to cover enough foreground and background pixels. Otherwise small objects in the image (such as diacritics) might be lost. However, choosing a block size that is too large can produce undesirable results if the image includes significant variations in illumination.

3.1.3 Normalizing Images. The reader pen is not always positioned perpendicular to the paper; the user might be tilting their hand when holding the pen. Therefore, words of text might be skewed, and it might be necessary to tilt them upward or downward a few degrees, to make them horizontal.

The de-skewing function expects to receive foreground objects as white pixels and background objects as black pixels. So, after thresholding the image to a binary format, the image is inverted using a bitwise NOT operation.

The calculation of the skew angle is then performed using the minimum bounding box method. A minimum bounding box has the smallest area that encloses all of the white pixels in an object [27]. There are different algorithms for finding the minimum bounding box. For the two-dimensional case, we need to find the smallest bounding rectangle.

Rotating calipers is an algorithm designed to solve many computational geometry problems including the bounding box [28]. After finding the bounding box, we calculate the angle of the box, which becomes the skewing angle. We then use a 2D rotation matrix and an affine transformation to normalize the image.

3.2 Optical Character Recognition (OCR)

In this section the process of Optical Character Recognition (OCR) is described. After preparing the text images, each binary image produced from previous step is processed by the OCR engine, which converts the images into equivalent text. The OCR software used is Tesseract OCR, which is open source and currently maintained by Google [29]. The OCR process is divided into steps, as shown in the flowchart in Figure 2. Figure 3 shows the pseudo code for our OCR algorithm.

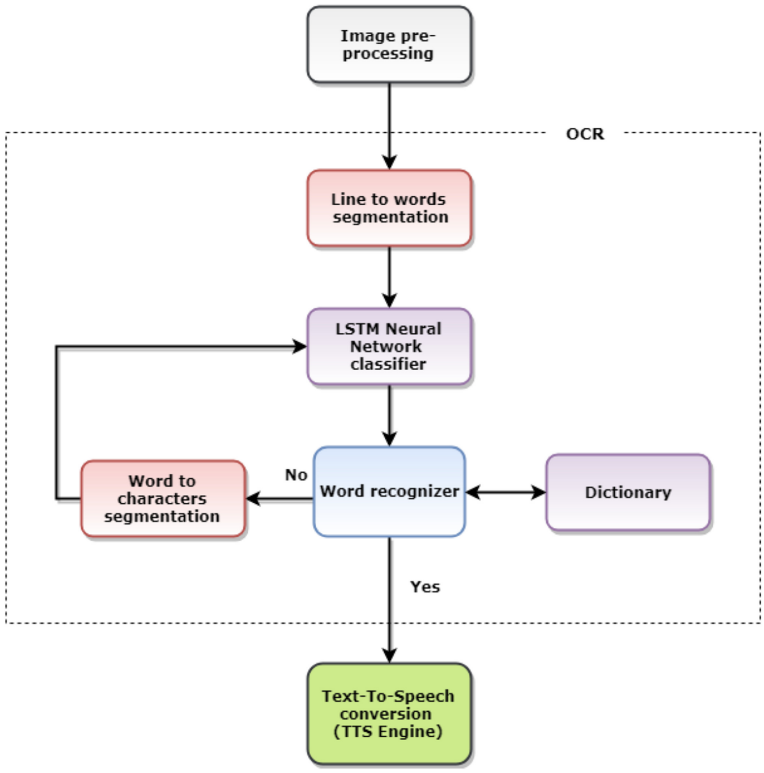


Fig. 2. Flowchart of OCR steps.

```
1:  Begin
2:      for each image
3:        find text blobs
4:          for each blob
5:            segment(line)
6:                    for each word in line
7:                      if (not recognize(word))
8:                        segment(word)
9:                          for each letter in word
10:                           recognize(letter)
11:                   end for
12:               end if
13:             end for
14:           end for
15:         end for
16: End
```

Fig. 3. Pseudo code of OCR process.

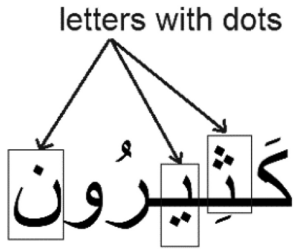


Fig. 4. Arabic letters with dots.

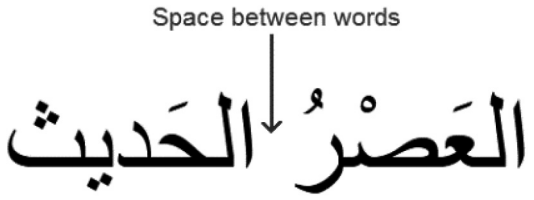


Fig. 5. Spacing between words in Arabic.

3.2.1 Word Segmentation. Arabic script is cursive, where many (but not all) characters in a word are connected together. That is challenging for the word segmenting process, not to mention the dots above and below certain characters, which are also considered part of the character's structure, as shown in Figure 4.

Strings of characters within the cursive script that are connected are called connected components. The first segmentation step finds connected components and stores the outlines of each connected component for the next step. Those outlines are then combined to form blobs, which include isolated characters adjacent to connected components as well as inner outlines or holes within the cursive script.

The blobs are then sequentially organized to represent the lines of the text. Using punctuations as a first delimiter and spacing as a second delimiter, these text lines are broken into words, An example of the spacing between words is shown in Figure 5.

Arabic script includes diacritics and letters with dots. Each of these is initially seen as a connected component with its own bounded box. For proper word segmentation, these boxes must be combined with boxes of the connected components that are above or below them. This is achieved by analyzing the size of the bounding box for each connected component and then sorting and labeling them according to size—as small or large. If a small, connected component exists too close to a large neighboring component, then a bounding box for both the small component and its larger neighbor is constructed. This ensures that the diacritics, and dots, are included within the bounding box containing the word, as shown in Figure 6.

3.2.2 Word Recognizer. Figure 7 shows the word recognizer. It includes a Static character classifier, an Adaptive character classifier, and a Character chopper. It uses the Character chopper in a recursive manner when necessary to chop words into characters.

The Character associator attempts to associate each blob with a word by comparing that blob to all the known words stored in the Dictionary. To do so, we used Tesseract OCR software [29], which employs best-first search to find the word with the lowest total distance rating.

If a satisfactory match for the word is not found in the Dictionary, the recognizer uses the Character chopper to iteratively chop the blob into individual characters, finding a first-best match

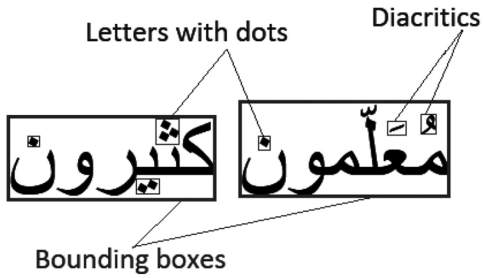


Fig. 6. Bounding boxes for diacritics, dots, and words.

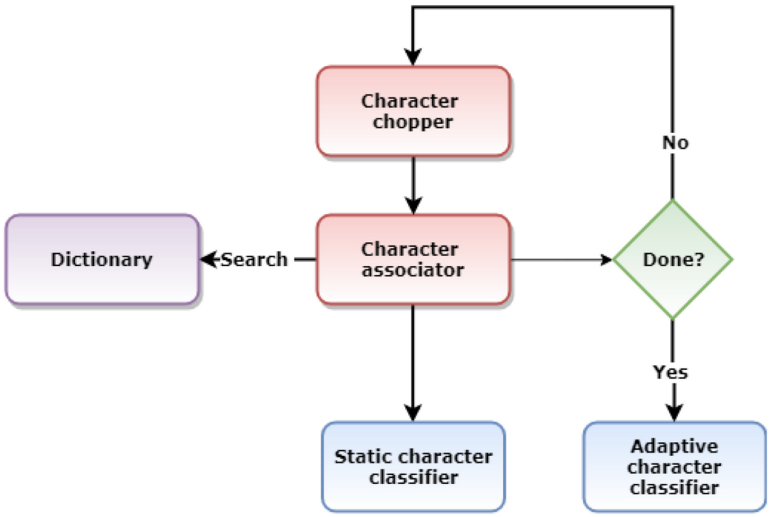


Fig. 7. Flowchart of word recognizer process.

for each character until it reaches the end of the blob. The characters (labeled with their best matches) are also passed to the Adaptive character classifier for training, so character classification results can improve over time.

Once the adaptive classifier has enough samples, it can provide classification results.

3.2.3 Character Segmentation. In this step, connected characters are separated using the Character chopper.

The bounding boxes of the connected components are used to construct a projection profile, which is a data structure that stores the number of projected pixels on x and y axes. That profile is then passed to a dynamic programming algorithm that is responsible for choosing the best set of segmentation points by calculating the cost. The cost function is the sum of profile entries at the cut points added to the variance of the spacing between the points.

The variance and the previously combined bounding boxes for small, connected components are combined, as shown in Figure 8.

3.2.4 Feature Vector Extraction. The words and characters are stored in the Dictionary as polygons. Thus, their features are extracted using a polygonal approximation method to represent the contours of these objects, as shown in Figure 9. The purpose is to capture the distinctive characteristics of the digitized characters and combine them into feature vectors. The polygonal

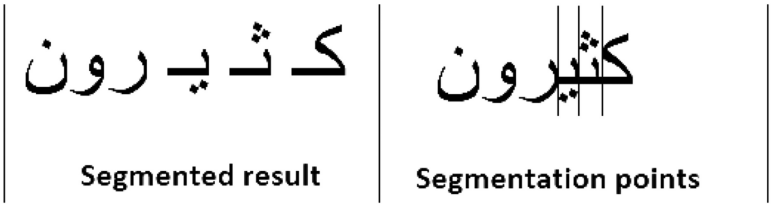


Fig. 8. Segmentation points and result.



Fig. 9. A polygonal approximation for an Arabic letter.

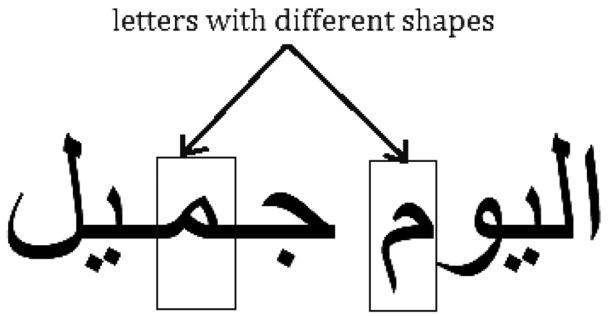


Fig. 10. An Arabic character with two different representations.

approximation technique is used in training, because it is invariant to the character size as well as translation and rotation.

In Arabic script, a single character might have up to four different representations, depending on its position within a word, as shown in Figure 10. This means that all these shapes must be mapped to the same classification.

3.2.5 Long Short-Term Memory Neural Network Classification. The method chosen for character classification is a Long Short-Term Memory (LSTM) neural network, which is a type of Recurrent Neural Network (RNN) [30]. RNNs have the form of a chain of repeating modules—each containing a neural network. In LSTM, each repeating module consists of a memory cell, an input gate, an output gate, and a forget gate.

The memory cell takes an input and stores it for some amount of time, which is similar to applying the identity function ($f(x) = x$) to the input. The derivative of the identity function is constant; thus, an LSTM network does not have the vanishing gradient problem.

The vanishing gradient problem is a difficulty found in training artificial neural networks with gradient-based learning methods and back propagation. When the neural network's weights receive updates proportional to the partial derivative of the error function, with respect to the current weight, during each iteration of training, each layer calculates the gradient with the effect of the gradient from the previous layer. The problem is that a small gradient will exponentially shrink as

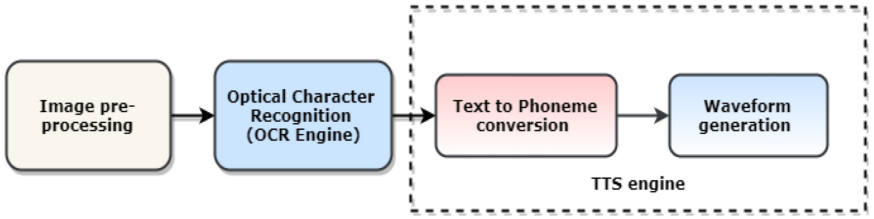


Fig. 11. The two main blocks in the TTS engine.

it approaches the earlier layers, effectively preventing the weights of those layers from changing value. This completely stops the neural network from further training [30].

The input gate of the LSTM controls the extent to which a new value flows into the cell. The forget gate controls the extent to which a value remains in the cell [32]. The output gate controls how the memory cell value affects the output of the LSTM unit. Gates are composed of a sigmoid neural net layer and a pointwise multiplication operation.

In this work, we used Tesseract OCR software V4.0.0 [29] that implements the LSTM Neural Network.

3.2.6 Arabic Text Diacritization. After the OCR Engine extracts the text from the scanned images, the resulting text lines are passed to the dicritization engine to enhance the Arabic speech. The engine used is Mishkal dicritization software, which is an open source Python project [31].

3.3 Text to Speech Conversion

After the extracted text is diacritized, the resulting text lines are passed to the Text-To-Speech (TTS) engine. For TTS, we used eSpeak NG [46], which is an open source engine that consists of two main units: a Natural Language Processing (NLP) unit and a Digital Signal Processing (DSP) waveform generator.

The NLP unit does the text-to-phoneme conversion, while the DSP unit generates the output speech waveform from the phoneme string. Figure 11 shows the high-level block diagram of the reader pen, highlighting the two main blocks in the TTS engine.

3.3.1 Natural Language Processing (NLP) Unit. The process flowchart of the NLP module is shown in Figure 12, and the NLP process flow is shown as pseudo code in Figure 13.

The NLP unit receives the input text stream from the OCR engine. First, a lexical analysis is performed to break the sentences into tokens by splitting the text at white spaces and punctuation marks—a process known as tokenization. (A token is a discrete element with an identified meaning.)

A parser then assigns a pronunciation to each token using rule-based analysis, with a dictionary for exceptions. The parser also determines the stress and emphasis positions. The result is then passed to the prosody processor.

The degree of naturalness of the output speech depends on prosodic factors. The Prosody processor determines the required intonation by assigning a proper amplitude and duration to each speech syllable, as well as the duration of pauses. Without prosody the output speech would sound monotonic and non-human like.

Phonetisation is the process of converting the parsed text into its equivalent phonemes. It is also known as grapheme-to-phoneme conversion. A phoneme is the smallest unit of speech sound. It is represented using phonetic notation from the International Phonetic Alphabet (IPA), which is

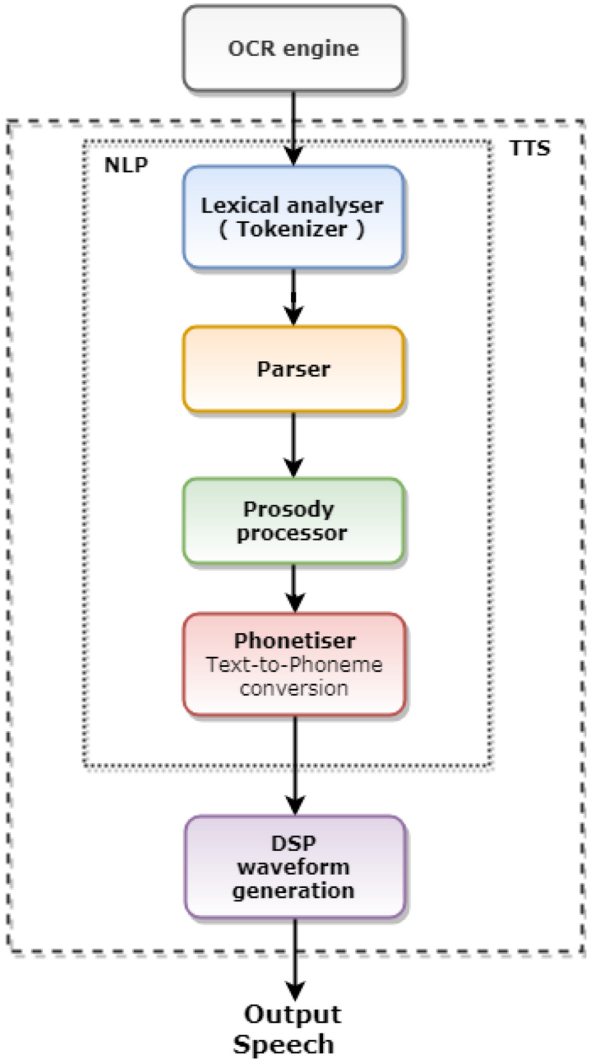


Fig. 12. Flowchart of NLP processing components.

an alphabetic system based on the Latin alphabet and is usually written between two slashes. Examples of an Arabic word and its phonetic representation are shown in Figure 14.

Phonemes are stored in a lookup table that contains all the phonemes used by a language, along with their properties, and data for their production, as sounds. Every character in a word is then mapped to its corresponding phoneme, and resulting phonemes are all joined together in a phoneme string. The phoneme string (with its prosody properties) is then passed to the DSP unit, which translates the phoneme string into a speech waveform.

3.3.2 The DSP Waveform Generator. The method we chose for speech synthesis is diphone concatenation. A diphone is the transition between two phones. Diphone concatenation puts together a string of previously recorded diphones to generate output speech. This method requires only a minimal speech database, containing one example of each diphone that occurs in

```
1:   Begin
2:   foreach text input
3:   sentences = split (text, punctuation-delimeters)
4:   for each sentence
5:   tokens = split(sentence, white-space)
6:   end for
7:   for each token
8:   phonemes = lookup(token)
9:   end for
10:  end for
11:  End
```

Fig. 13. Pseudocode of the NLP unit.

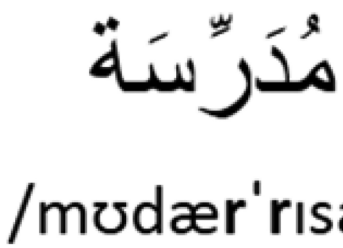


Fig. 14. An Arabic word and its phonetic representation.

a language. Arabic has 38 phones, so the number of diphones is the number of phones squared, which is 1,444 diphones [33].

The DSP waveform generator uses the string of phonemes that it receives from the Text-to-Speech conversion to look up diphones in the Diphone database, as shown in the flowchart in Figure 15.

That diphone stream is then passed to the prosody processing unit, which applies the string of appropriate prosody properties (also received from the NLP unit) to each position within the diphone stream. The resulting diphone string is then passed to the concatenation unit, which produces the speech signal.

The diphone database used in this research was provided by Reference [34]. The database memory size is 6.564 Megabytes. It consists of diphone labels, waveforms, durations, and border positions between phones. The speech used in creating the diphone database must be recorded in a monotonic manner to allow the prosody processor to modulate the intonation of speech as desired.

Diphone synthesis has the advantage of a manageable database size. However, it suffers from sonic glitches, due to discontinuities when concatenating diphone signals. This can result in a robotic-sounding output. To overcome this problem, the "glitchy" synthesized speech signal is passed through a signal smoothing step, called the Multi-Band Resynthesis OverLap-Add (MBROLA) algorithm.

MBROLA is a time-domain algorithm that works by dividing the speech waveform into small segments and then combining those segments with an overlap-add technique [35]. The Overlap-Add method is an alternative to sinusoidal summation of signals, which is more computationally expensive [36].

Due to limited resources and processing power usually found in embedded devices, this work used the MBROLA light weight method. We used the pre-defined Prosody models and the default values for all input parameters (such as phone duration and F0).

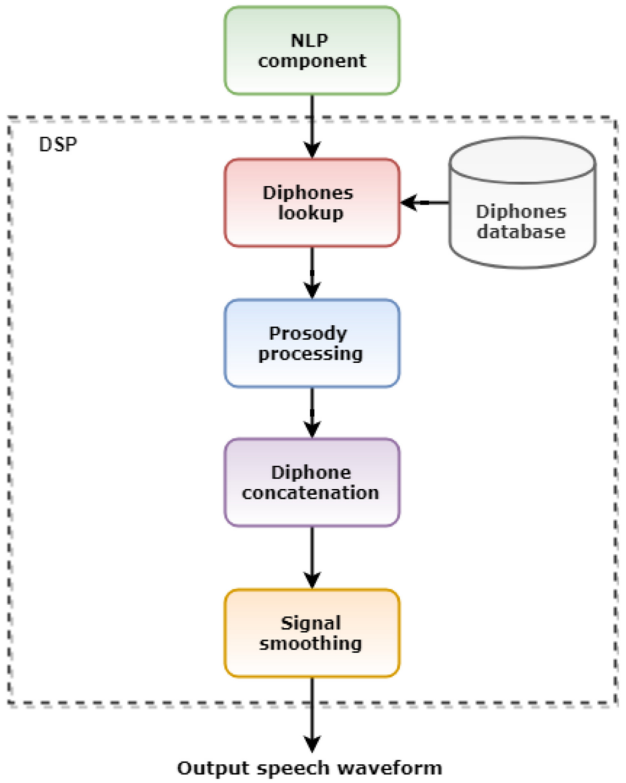


Fig. 15. Flowchart of DSP waveform generator.

4 HARDWARE COMPONENTS

This section provides a description of the hardware components that are used in the prototype of the reader pen, as shown in the hardware block diagram of Figure 16.

We chose a Raspberry Pi Zero as a processor for this system, because (1) it can perform complex tasks that involve neural networks, which are needed for optical character classification, and (2) it runs an operating system, so it can execute tasks that require file manipulation.

There are many models and boards available for the Raspberry Pi. We chose to work with Raspberry Pi Zero because of its small chip size and because its hardware specifications meet our system requirements. Moreover, the Raspberry Pi Zero can read a micro SD card. Therefore, the OCR and TTS software systems were embedded to the Raspberry board by installing them on the SD card. A Raspberry Pi Zero board is shown in Figure 17 [37].

The Raspberry Pi Zero specifications are as follows:

- Processor specifications: 1 GHz single-core CPU.
- Memory specifications: 512 MB RAM.
- Ports: Mini HDMI port, micro USB OTG port, micro USB power.
- Camera support: Composite video and reset headers and a CSI camera connector (v1.3 only).

There are many operating systems that work with Raspberry Pi, such as Raspbian Jessie and Minibian—both Debian-based Linux operating systems, as well as Windows 10 IoT Core operating system [38]. During the development phase, we chose to work with Raspbian Jessie, because it

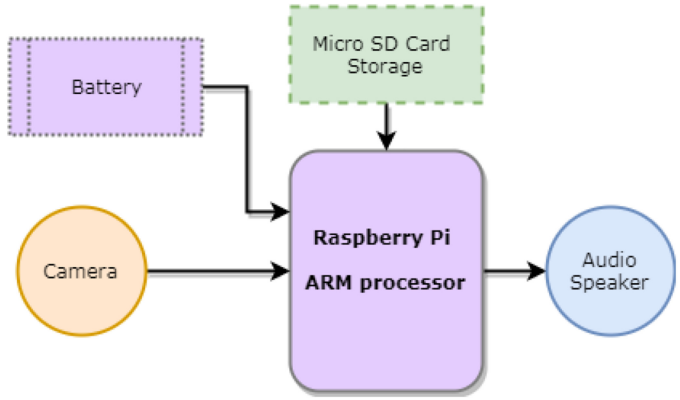


Fig. 16. TTS hardware components block diagram.

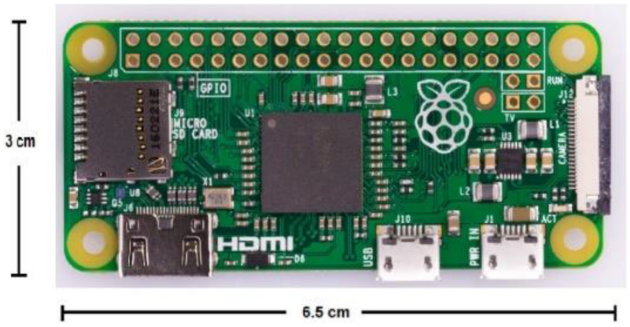


Fig. 17. Raspberry Pi Zero chip architecture.

has a convenient graphical user interface (GUI) that makes the development of the code easier, with the help of an Integrated Development Environment (IDE). However, a minimized operating system (without a GUI) would be more practical for production, since the GUI requires considerable memory and CPU resources. Freeing those resources would enhance the performance of the program. (Raspbian Jessie Lite is an example of such a minimized operating system.)

The camera used for capturing the text images is a Raspberry Pi camera module, version 2. This camera has a resolution of 8 megapixels, a rolling shutter, and a fixed focus. Since the reader pen will be used to capture really closeup images, the camera focus had to be adjusted using a lens adjustment tool.

The camera is also connected to an infrared-based obstacle detection sensor, which consists of an infrared transmitter and receiver. The infrared transmitter emits an IR signal and, when an obstacle appears, that object reflects the IR light into the receiver. The receiver then signals the camera to start capturing. (The infrared sensing system has an adjustable detection distance.)

To make the reader pen portable, it had to be connected to a battery. However, it is not easy to find a battery that is small, affordable, and capable of providing the power requirements for the Raspberry Pi [39]. The battery we found closest to the requirements is a lithium polymer battery with a voltage of 3.7 volts. A boost circuit is needed to boost that voltage to 5 volts. There are several different models and manufacturers for this boost circuit [40].

Choosing the type of audio output for the Raspberry Pi Zero was difficult, given the fact that it does not have a sound card or even a standard audio jack output. However, the Raspberry Pi

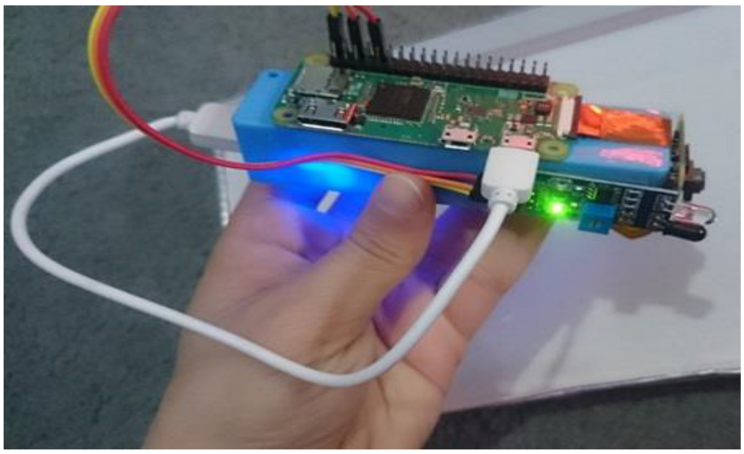


Fig. 18. The reader pen prototype.

Zero W model has a Bluetooth module, which allows it to connect to a Bluetooth headphone. This increases the portability of the device. Figure 18 shows a photo of the prototype.

5 EXPERIMENTAL EVALUATION

This section describes the experimental methods used to evaluate the OCR (including image pre-processing) and the TTS modules. Our aim was to evaluate the performance of the reader pen in recognizing text and producing speech and make sure that the pen meets the real-time constraints. At first, each of those two modules is evaluated separately. Then, a more general real-time usability testing is performed to provide the final experimental evaluation.

The evaluation tests were performed on a Raspberry Pi Zero W with a single-core 1 GHz CPU, 512 MB RAM, and the Raspbian Jessie operating system.

5.1 Evaluation of OCR (Including Image Pre-processing)

To evaluate the effectiveness of the OCR program (including image pre-processing) experimentally, a set of metrics was used, along with a dataset of Arabic text images. The evaluation focused on two aspects: the accuracy and the response time of the OCR.

The dataset used for testing must be general enough to provide a useful measurement. Arabic Printed Text Image (APTI) [41] is a public dataset that is widely used for evaluating Arabic OCR. It is a word-level dataset, where each text image contains only one Arabic word. This is similar to what a reader pen camera will capture. The APTI text images contain 10 different Arabic fonts, with four styles for each font (plain, italic, bold, and bold italic), as well as different font sizes, from 6 to 24 points. The average image dimension is around 48×48 pixels, depending on the font size.

The first metric used was the overall character accuracy for all the tested images, which is the percentage of ground-truth characters that are recognized correctly in an Arabic text image. It is evaluated by comparing the ground-truth text file to the OCR output text file. The character accuracy is calculated by Equation (2):

$$Overall\ character\ accuracy = \frac{m - c}{m} \times 100 \tag{2}$$

where c is the number of characters that are recognized incorrectly, and m is the total number of characters in the reference text.

Table 1. Quality of Speech MOS Scale

Quality of speech	Score
Excellent	5
Good	4
Fair	3
Poor	2
Bad	1

Table 2. Listening Effort MOS Scale

Effort required to understand the meanings	Score
Complete relaxation possible, no effort required	5
Attention necessary, no appreciable effort required	4
Moderate effort required	3
Considerable effort required	2
No meaning understood with any feasible effort	1

The second metric is overall word accuracy for all the tested images, which is the percentage of words that are recognized correctly, which is calculated using Equation (3):

$$Overall\ word\ accuracy = \frac{w - e}{w} \times 100 \tag{3}$$

where e is the number of words that are recognized incorrectly, and w is the total number of words in the reference samples.

The third performance metric is the average response time of the OCR engine. One recent study compared the most common Arabic OCR systems, based on the impact that the quality of the input images (i.e., image resolution) had on the accuracy of these OCR systems [42]. However, the quality of the input images also affects the response time of the OCR. Our experiment tests the response time of the OCR engine for camera-captured images and digital images, over a range of different image sizes.

5.2 Evaluation of TTS

In general, there are two methods to evaluate speech quality: (1) the subjective approach, where test subjects are asked to judge the quality of the speech that they hear, and (2) the objective approach, where equipment is used to measure the physical properties of the speech signal [43].

The subjective approach is more time-consuming. However, it is more commonly used, because the end user of a TTS-system is a human listener. Current objective methods (such as signal-to-noise ratio) are better suited to voice communication systems, where the received speech signal can be compared to the original speech signal. No original signal exists when synthesizing speech from text [44].

The Mean Opinion Score (MOS) is a subjective test that has been standardized by the International Telecommunication Union (ITU) and is widely used to measure audio and video quality. It is most commonly used for evaluating synthesized speech quality. Therefore, the MOS test was chosen to evaluate the speech quality of the reader pen [45].

The MOS result is a rational number in the range 1–5, where 1 is lowest quality and 5 is the highest quality. The MOS scale for speech quality is shown in Table 1, while Table 2 shows the scale for rating the listening effort, and Table 3 shows the scale for rating speech loudness [45].

Table 3. Loudness-preference MOS Scale

Loudness preference	Score
Much louder than preferred	5
Louder than preferred	4
Preferred	3
Quieter than preferred	2
Much quieter than preferred	1

The MOS is the arithmetic mean over a series of N individual ratings (R1 - Rn) provided by subjects, as they listen to a set of N audio stimuli during a subjective quality evaluation test (see Equation (4)):

$$MOS = \frac{\sum_{n=1}^{N} Rn}{N} \tag{4}$$

ITU recommends the use of seven different questions to evaluate: Listening Effort, Comprehension Problems, Voice Naturalness/Humanlike Voice, Voice Quality, Loudness, Intonation, and Confidence.

The individual speech samples should consist of random simple meaningful sentences. They should be easy to understand, and they should not include technical or scientific terms. The test listeners can be any native speaker of the language, with no hearing disability. The recommended number of listeners should be between 6 and 40 [45].

The reader pen MOS experiment was conducted using an online audio database consisting of seven speech samples that were all synthesized by the TTS software of the reader pen and seven survey questions for each speech sample. The seven survey questions are shown in Table 4. The total number of subjects was 40 listeners, the same seven samples were played for each listener.

5.3 Real-time Usability Testing and Evaluation

The purpose of this usability study is to ensure that the proposed Reader Pen is usable and useful, as determined in real-time by a set of target end-users. It measures user satisfaction, in terms of ease of use, and user-friendliness. It also provides design recommendations for the next version of the prototype.

The Higher Council for the Rights of Persons with Disabilities (HDC) introduced us to people with visual disabilities. Ten people participated in our real-time usability testing for the proposed reader pen assistive device. Their task was to read a text using the pen. The subjects were given a tutorial on how to use the pen. They were also asked to comment on the device by filling a usability evaluation form orally. The evaluation sheet is shown in Table 5 [47–49].

The instructions given to the subjects were as follows:

- You are requested to read the following text using our proposed assistive device “Reader Pen.”
- The Pen will read every word in the text and convert it to speech.
- You are then requested to evaluate the Pen, using the following evaluation sheet. Please indicate, on a scale from 1 to 5, how well you agree with each of the following statements. A score of “5” in the form means that you “strongly agree” with the statement, while a score of “1” means that you “strongly disagree” with the statement.

Table 4. Speech Quality Survey

Q1. Please rate the hearing effort
Complete relaxation possible, no effort required
Attention necessary, no appreciable effort required
Moderate effort required
Considerable effort required
No meaning understood with any feasible effort
Q2. Please rate the speech quality
Excellent
Good
Fair
Poor
Bad
Q3. Please rate the speech intonation
Excellent
Good
Fair
Poor
Bad
Q4. Please rate the speech volume level
Much louder than preferred
Louder than preferred
Preferred
Quieter than preferred
Much quieter than preferred
Q5. Please rate the speech naturalness/human like
Excellent
Good
Fair
Poor
Bad
Q6. Please evaluate the speaking rate
Excellent
Good
Fair
Poor
Bad
Q7. Please rate the words pronunciation
Excellent
Good
Fair
Poor
Bad

Table 5. Usability Study Evaluation Sheet

#	Item	Score
1	**Understanding:** users can understand the delivered speech of the captured Arabic text.	1 2 3 4 5
2	**Ease of use:** the pen is easy to use.	1 2 3 4 5
3	**Design:** the overall design and look of the pen is neat and comfortable.	1 2 3 4 5
4	**Control:** users have control over device operations (capturing text, delivering speech, etc.)	1 2 3 4 5
5	**Confidence:** users are confident in both the pen and their ability to use it.	1 2 3 4 5
6	**Real-time:** the response time of the pen (in delivering the speech) is fast and suitable.	1 2 3 4 5
7	**Engaging:** overall, the product is pleasant and satisfying to use.	1 2 3 4 5
8	**Unwanted Attention:** as an assistive device, this pen would NOT draw unwanted attention.	1 2 3 4 5
9	**Idea:** the idea behind the device provides a good solution to enable people with visual and/or learning disabilities to access printed Arabic material in real time.	1 2 3 4 5
10	**Purchase:** users are willing to buy and use this pen, if it was available for sale.	1 2 3 4 5
Additional Comments / Feedback:		

6 RESULTS AND DISCUSSION

In this section, we present and discuss the results of our experiments regarding the OCR accuracy, the OCR response time, the speech quality, the real-time constraints, and the usability study.

6.1 The OCR Accuracy

Figure 19 shows the overall character accuracy percentage for nine different Arabic fonts in plain style in five different font sizes. The fonts were selected because they are most commonly used in Arabic textbooks, due to these fonts being provided in Microsoft Word by default. These overall accuracies were calculated across the 113,284 words in the APTI database.

Figure 20 shows the overall word accuracy percentage for nine different Arabic fonts in plain style in five different font sizes. The overall accuracy was calculated across the 113,284 words in the APTI database.

These accuracy results show that the fonts with the highest character and word accuracy percentage are the Simplified Arabic and the Arabic Transparent fonts. These are the most commonly used fonts in books and newspapers. However, the accuracy results are low for some fonts, such as Andalus, DecoType Naskh, and DecoType Thuluth. The training examples used to train the system were Arabic pages taken randomly from books and newspapers. Some Arabic fonts are not easy to read and are less popular. Therefore, they occurred less frequently in our random training dataset. The result was a much lower recognition accuracy for those fonts.

Figure 21 shows the overall character accuracy percentage in four different image sizes. The overall accuracy was calculated across the 113,284 words in the APTI database for Simplified Arabic font.

6.2 The OCR Response Time

The response times for different image sizes are shown in Figure 22.

The results in Figure 22 show that the response time of the OCR program (including image preprocessing) is affected by the quality (i.e. the resolution) of the input images. That response time

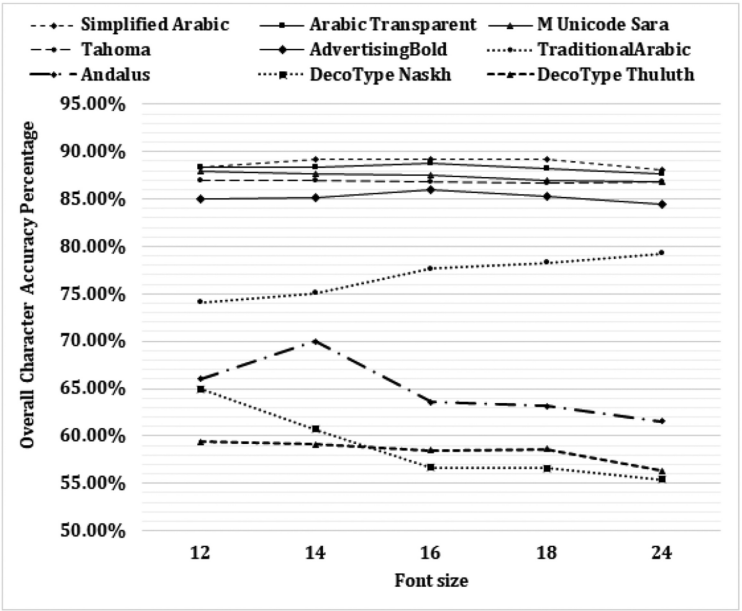


Fig. 19. Font size versus overall character accuracy.

increases with increasing image dimensions, because larger images fill more memory and require more processing at runtime. According to the results in Figure 21 and Figure 22, the chosen image size for the reader pen is in between 490 × 250 and 750 × 350 to obtain a reasonable response time and an acceptable accuracy.

6.3 Speech Quality

The MOS results are shown in Table 6. These results show that most of the features of the speech were judged to be in the Fair–Good range, and users were able to understand the speech. The system achieved an overall MOS score of about 3.54, which is better than the results shown in previous related works. However, the average MOS score for speech naturalness was relatively low, compared to the rest of the scores.

Future work will focus on enhancing the MOS results of the TSS module using state-of-the-art TTS, including Arabic statistical parametric speech synthesis.

6.4 Meeting Real-time Constraints

We have designed our Arabic Reader Pen system to be able to meet real-time constraints, which means the system should meet the specified deadline, regardless of the workload. As mentioned in Section 3, the proposed reader system algorithm consists of three main processes: image pre-processing, optical character recognition (OCR engine), and Text-To-Speech conversion (TTS engine). The evaluation of real-time constraints for the OCR module (including image pre-processing) is demonstrated in Section 6.2, where the response time is affected by the image size and quality, as well as other parameters. Keeping these other parameters fixed and adjusting the image size and quality before passing the image to the OCR will ensure that the software meets the real-time constraint. Based on the results of Figure 21 and Figure 22, the optimal image size for the reader pen system to meet the real-time constraints, with an acceptable accuracy, is between 490 × 250 and 750 × 350. Thus, we chose an image size of 640 × 480 for real-time usability testing. As for

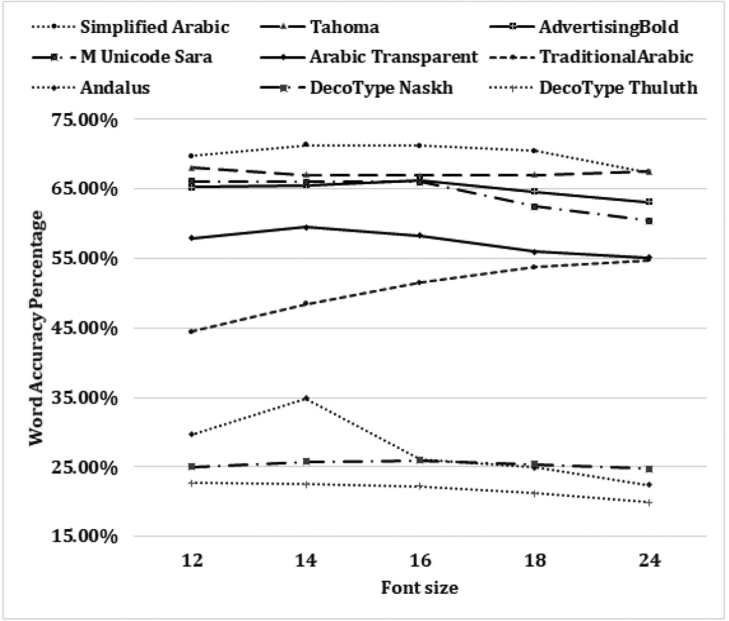


Fig. 20. Font size versus overall word accuracy.

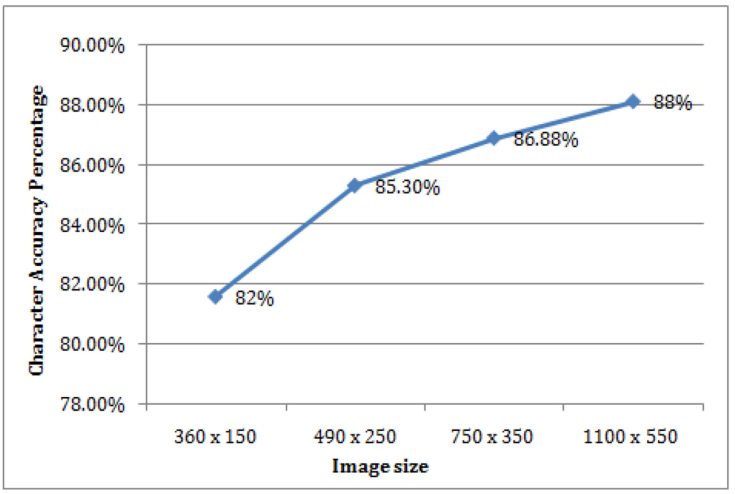


Fig. 21. Image size versus overall character accuracy.

the TTS module, we used the MBROLA algorithm, which is designed and implemented to meet real-time constraints [35].

6.5 The Usability Study Report

Ten people participated in the real-time usability testing and gave us their feedback about the usability of the proposed Arabic reader pen.

Table 7 summarizes the evaluation collected during the real-time testing of our reader pen assistive device. It presents the average score and standard deviation for each of the 10 statements

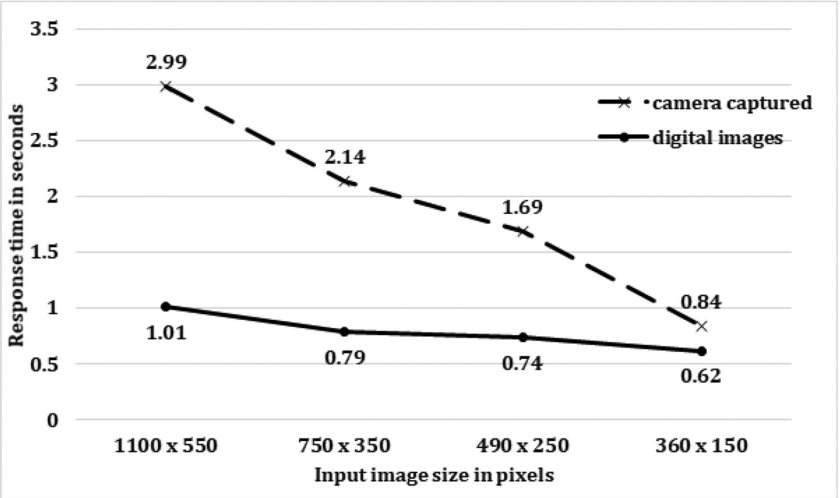


Fig. 22. The effect of image size on the response time.

Table 6. MOS Results

	Question	Score
1	Hearing effort	3.375
2	Speech quality	3.65
3	Speech intonation	3.525
4	Naturalness Humanlike	2.95
5	Pronunciation	3.875
6	Speaking rate	4.15
7	Speech volume	3.225
Final MOS score out of 5		3.5357

in the evaluation sheet (shown in Table 5) across the 10 participants as they used our reader pen in real-time testing. Figure 23 summarizes these results visually using a bar graph.

The results presented in Table 7 and Figure 23 show that items 1 (*Understanding*), 2 (*Ease of use*), 5 (*Confidence*), 6 (*Real-time*), 7 (*Engaging*), 9 (*Idea*), and 10 (*Purchase*) received relatively high scores of agreements across the 10 participants, with an average score greater than or equal to 4. However, items 3 (*Design*), 4 (*Control*), and 8 (*Unwanted attention*) received relatively low scores of agreements across the 10 participants, with an average score less than or equal to 3. The highest score received was for items 2 (*Ease of use*) and 6 (*Real-time*), while the lowest score received was for item 4 (*Control*). The overall average across the 10 statements is 3.9.

Moreover, the participants left the following comments and feedback on the evaluation sheets:

- The pen is helpful for users who have low vision and users with reading difficulties such as Dyslexia.
- The reader pen is difficult to use by blind users because they are unable to locate the words for the pen to scan it.
- The size of the pen prototype needs to be smaller and lighter weight.
- The speaking rate and volume should be a user setting, so each user can change it to their preference.

Table 7. Usability Study Results Summary

#	Item	Average Score	Standard Deviation
1	**Understanding:** users can understand the delivered speech of the captured Arabic text.	4.1	0.7
2	**Ease of use:** the pen is easy to use.	4.5	0.5
3	**Design:** the overall design and look of the pen is neat and comfortable.	3	0.4
4	**Control:** users have control over device operations (capturing text, delivering speech, etc.)	2.8	0.4
5	**Confidence:** users are confident in both the pen and their ability to use it.	4.2	0.4
6	**Real-time:** the response time of the pen (in delivering the speech) is fast and suitable.	4.5	0.5
7	**Engaging:** overall, the product is pleasant and satisfying to use.	4.3	0.5
8	**Unwanted attention:** as an assistive device, this pen would NOT draw unwanted attention.	2.9	0.3
9	**Idea:** the idea behind the device provides a good solution to enable people with visual and/or learning disabilities to access printed Arabic material in real time.	4.4	0.5
10	**Purchase:** users are willing to buy and use this pen if it was available for sale.	4	0.6

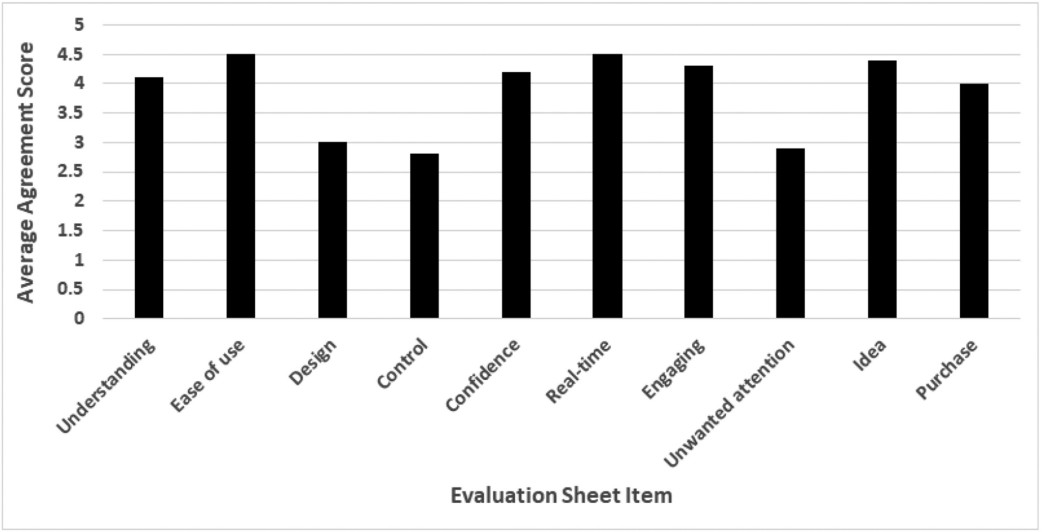


Fig. 23. Usability study results summary.

- The speech quality is considered slightly robotic compared to the Voice Over and Talkback screen readers used in iPhone and Android platforms.

The above recorded results suggest that the proposed Arabic reader pen is easy to use and that the users can feel confident while using it. The results also suggest that the pen is useful in reading

and understanding printed Arabic text and provides a good solution to enable people with visual and/or learning disabilities to access printed Arabic material in real-time. Of most importance, the proposed pen is fast and suitable in delivering the speech, and thus meets the real-time constraints. Finally, the users would be willing to buy and use the proposed pen if it was available for sale.

However, the results suggest that more work is needed in the overall design and look of the pen. At least two aspects of the design should be considered. First, the pen should not draw any unwanted attention to its users. For example, its size should be smaller, and its weight should be lighter. Second, the pen should provide more control to the users over its operations, such as speaking rate and volume.

Taken together, with an overall average score of agreements of 3.9, the proposed Arabic reader pen is pleasant and satisfying to use. The results of this device can contribute to make printed Arabic material accessible to visually impaired persons and people with learning disabilities.

7 CONCLUSION AND FUTURE WORK

In this article, we proposed to answer the following the research question:

Q: How could we build a real-time assistive embedded system that serves as a Text-To-Speech reader pen for the Arabic language?

A: We developed a real-time assistive reader pen for Arabic that employs Optical Character Recognition (OCR) and Test-To-Speech (TTS) conversion to allow people with visual disability to access printed Arabic text. The system employs LSTM neural networks for optical character recognition, and Arabic speech synthesis is achieved through concatenation synthesis, followed by the MBROLA algorithm. The performance of the OCR module achieved an accuracy of 88%. The performance of the TTS module achieved a Mean Opinion Score (MOS) of 3.53 (out of 5)—indicating that the users were able to understand the delivered speech. The current prototype of the Arabic reader pen was used in real-time usability testing with 10 subjects. Overall, the results presented in this article suggest that this prototype was judged to be helpful by people with low vision.

As for future work, the current prototype of the proposed Arabic reader pen assistive device has room for enhancement in the following areas:

- Enhancing the OCR accuracy for some Arabic fonts by adding training data for these fonts and adding text correction stage after the OCR.
- Enhancing the human-like factor in the TTS module.
- Improving the MOS results of the TTS module by using state-of-the-art TTS, including Arabic statistical parametric speech synthesis.
- Evaluating the TTS module objectively by comparing phone duration, F0, and energy values with the target ones.
- Evaluating the TTS module subjectively, through Degradation MOS (DMOS) and preference tests.
- More work will be done on the overall design and look of the Pen to reduce its size and weight.
- The next version of the prototype will provide more control to the users over its operations, such as speaking rate and volume.

REFERENCES

[1] World Health Organization. 2001. International classification of functioning, disability and health ICF. *World Health Organization*.

[2] World Health Organization. 2014. Fact sheet no. 352, 2014.

[3] World Health Organization. 2014. Visual impairment and blindness fact sheet N 282. *World Health Organization* 2014.

[4] J. Taylor. 2018. Educating students with visual impairments for inclusion in society. *Amer. Found. Blind*, 2000. Retrieved from http://www.afb.org/info/teachers/inclusive-education/35.

[5] T. Cavanaugh. 2002. The need for assistive technology in educational technology. *AACE Rev.* 10, 1 (2002), 27–31.

[6] J. Allen. 1979. MITalk-79: The 1979 MIT text-to-speech system. *J. Acoust. Soc. Amer.* 65, S1 (1979).

[7] N. N. Akhlagi, F. Lonn, and P. Wittrup. 2003. Reading pen. United States of America Patent 6, 509 893, 21 2003.

[8] K. C. Ray and A. Rawoof. 2014. ARM based implementation of text-to-speech (TTS) for real time embedded system. In *International Conference on Signal and Image Processing (ICSIP'14)*.

[9] S. A. Sanaki and B. B. S. 2015. Embedded based implementation of real time text-to-speech conversion. *Int. J. Res.* 2, 8 (2015), 339–345.

[10] M. Hamad and M. Hussain. 2011. Arabic text-to-speech synthesizer. In *IEEE Student Conference on Research and Development (SCOReD'11)*.

[11] P. K. Bamini. 2003. *FPGA-based Implementation of Concatenative Speech Synthesis Algorithm* 2003.

[12] H. Tora, İ. B. Uslu, and T. Karameh. 2017. Implementation of Turkish text-to-speech synthesis on a voice synthesizer card with prosodic features. *Anadolu Univ. J. Sci. Technol. A- Appl. Sci. Eng.* 18, 3 (2017).

[13] RC Systems. 2006. DoubleTalk RC8660, 23 Mar 2006. Retrieved on December 2020 from https://www.rcsys.com/Downloads/rc8660.pdf.

[14] A. Chabchoub and A. Cherif. 2011. High quality Arabic concatenative speech synthesis. *Sig. Image Proc. Int. J.* 2 (2011).

[15] A. W. Black. 2002. Perfect synthesis for all of the people all of the time. In *IEEE Workshop on Speech Synthesis*.

[16] J. Bachan and M. Tokarski. 2017. Creation and evaluation of MaryTTS speech synthesis for polish. In *Language and Technology Conference*.

[17] K. P. Sarathy and A. G. Ramakrishnan. 2008. Text to speech synthesis system for mobile applications. In *Workshop in Image and Signal Processing (WISP'08)*.

[18] E. Vanitha, P. K. Kasarla, and E. Kuamarswamy. 2015. Implementation of text-to-speech for real time embedded system using Raspberry Pi processor. *Int. J. Mag. Eng. Technol. Manag. Res.* 2, 7 (2015).

[19] I. Rebai and Y. BenAyed. 2016. Arabic speech synthesis and diacritic recognition. *Int. J. Speech Technol.* 19, 3 (2016), 485–494.

[20] D. Frontini and M. Malcangi. 2006. Neural network-based speech synthesis. In *DSP Application Day*.

[21] K. Lakshmi and T. C. S. Rao. 2016. Design and implementation of text to speech conversion using Raspberry Pi. *Int. J. Innov. Technol. Res.* 4, 6 (2016).

[22] P. Fogarassy-Neszly and C. Pribeanu. 2016. Multilingual text-to-speech software component for dynamic language identification and voice switching. In *International Conference on Human-computer Interaction*.

[23] Y. Wang, R. Skerry-Ryan, D. Stanton, Y. Wu, R. J. Weiss, N. Jaitly, Z. Yang, Y. Xiao, Z. Chen, S. Bengio, Q. Le, Y. Agiomyrgiannakis, R. Clark, and R. A. Saurous. 2017. Tacotron: Towards end-to-end speech synthesis. In *Interspeech*. 4006–4010.

[24] Yu Zhang, Ron Weiss, Heiga Zen, Yonghui Wu, Zhifeng Chen, R. J. Skerry-Ryan, Ye Jia, Andrew Rosenberg, and Bhuvana Ramabhadran. 2019. Learning to speak fluently in a foreign language: Multilingual speech synthesis and cross-language voice cloning. In *Interspeech*. 2080–2084. Retrieved from 10.21437/Interspeech.2019-2668.

[25] B. Phil, S. Polansky, D. Repetto, M. Roberts, and D. Rockmore. 2011. Music and computers: A theoretical and historical approach. Preface to the Archival Version.

[26] S. Lukose and S. S. Upadhya. 2017. Text to speech synthesizer-formant synthesis. In *International Conference on Nascent Technologies in Engineering (ICNTE'17)*.

[27] G. Toussaint. 1983. Solving geometric problems with the rotating calipers. In *IEEE MELECON'83*.

[28] M. I. Shamos. 1978. *Computational Geometry*, Yale University.

[29] R. Smith. 2007. An overview of the tesseract OCR engine. In *9th International Conference on Document Analysis and Recognition (ICDAR'07)*. 629–633. Retrieved from 10.1109/ICDAR.2007.4376991.

[30] S. Hochreiter and J. Schmidhuber. 1997. Long short-term memory. *Neur. Comput. J.* 9 (1997).

[31] T. Zerrouki. 2014. Mishkal diacritiser. Retrieved from https://github.com/linuxscout/mishkal.

[32] F. A. Gers, J. Schmidhuber, and F. Cummins. 1999. Learning to forget: Continual prediction with LSTM. *Neural Comput.* 12 (1999).

[33] S. H. Al-Ani. 2014. *Arabic Phonology: An Acoustical and Physiological Investigation*. Walter de Gruyter.

[34] Faculte Polytechnique de Mons - TCTS lab. 1998. MBROLA voices project at Github. Retrieved from https://github.com/numediart/MBROLA-voices/tree/master/data/ar2.

[35] T. Dutoit, V. Pagel, N. Pierret, and F. Bataille. 1996. The MBROLA project: Towards a set of high quality speech synthesizers free of use for non commercial purposes. In *International Conference on Spoken Language Processing (ICSLP'96)*.

[36] M. H. Hayes. 1998. *Schaum's Outline of Digital Signal Processing*. McGraw-Hill.

[37] N. Health. 2018. Raspberry Pi Zero W: The smart person's guide. *TechRepublic,* 2018. Retrieved from https://techrepublic.com/article/raspberry-pi-zero-wireless-the-smart-persons-guide/.
[38] M. Gibbs. 2018. Ten operating systems for the Raspberry Pi. *Netw. World* 3 Nov. (2014). Retrieved from https://networkworld.com/article/2842678/computers/ten-operating-systems-for-the-raspberry-pi.html.
[39] Raspi TV. 2017. How much power does pi zero w use? Retrieved from http://raspi.tv/2017/how-much-power-does-pi-zero-w-use.
[40] Adafriut. Adafriut POWERBOOST 500 CHARGER. Retrieved on December 2020 from https://adafruit.com/product/1944.
[41] F. E. A. Slimane. 2009. A new Arabic printed text image database and evaluation protocols. In *10th International Conference on Document Analysis and Recognition (ICDAR'09)*.
[42] S. Saber, A. Ahmed, A. Elsisi, and M. Hadhoud. 2016. Performance evaluation of Arabic optical. In *International Conference on Advanced Intelligent Systems and Informatics (AISI'15)*.
[43] V. Grancharov and W. Kleijn. 2008. Speech quality assessment. In *Springer Handbook of Speech Processing*, Berlin, Springer, 83–100.
[44] W. B. Kleijn and K. K. Paliwal. 1995. Quality evaluation of synthesized speech. In *Speech Coding and Synthesis*, Elsevier Science Inc., 709–734.
[45] International Telecommunication Union. 1996. Recommendation P.800, ITU, 1996. Retrieved on December 2020 from https://www.itu.int/rec/T-REC-P.800-199608-I.
[46] eSpeak. 2020. eSpeak NG Text-To-Speech. GitHub, Inc. Retrieved from https://github.com/espeak-ng/espeak-ng.
[47] M. A. Alzubaidi and M. Otoom. 2018. Discussion-facilitator: towards enabling students with hearing disabilities to participate in classroom discussions. *Int. J. Technol. Enhanc. Learn.* 10, 1–2 (2018), 73–90.
[48] M. Otoom and M. A. Alzubaidi. 2018. Ambient intelligence framework for real-time speech-to-sign translation. *Assist. Technol.* 27, 30 (2018), 119–132.
[49] M. Otoom, M. A. Alzubaidi, and R. Aloufee. 2020. Novel navigation assistive device for deaf drivers. *Assist. Technol.* 2020 10 (2020), 1–1.
[50] T. Zerrouki, M. M. A. Shquier, A. Balla, N. Bousbia, I. Sakraoui, and F. Boudardara. 2019. Adapting eSpeak to Arabic language: Converting arabic text to speech language using eSpeak. *Int. J. Reas.-based Intell. Syst.* 11, 1 (2019), 76–89.
[51] Imene Zangar, Zied Mnasri, Vincent Colotte, Denis Jouvet, and Amal Houidhek. 2018. Duration modeling using DNN for Arabic speech synthesis. In *9th International Conference on Speech Prosody*.
[52] O. Zine and A. Meziane. 2017. Novel approach for quality enhancement of Arabic text to speech synthesis. In *International Conference on Advanced Technologies for Signal and Image Processing (ATSIP'17)*. IEEE, 1–6.
[53] O. Zine, A. Meziane, and M. Boudchiche. 2017. Towards a high-quality lemma-based text to speech system for the Arabic language. In *International Conference on Arabic Language Processing*. Springer, Cham, 53–66.
[54] Amrouche Aissa, Leila Falek, and Hocine Teffahi. 2017. Design and implementation of a diacritic Arabic text-to-speech system. *Int. Arab J. Inf. Technol.* 14, 4 (2017).
[55] Abdelali Ahmed, Mohammed Attia, Younes Samih, Kareem Darwish, and Hamdy Mubarak. 2018. Diacritization of Maghrebi Arabic sub-dialects. *ArXiv Preprint arXiv:1810.06619* (2018).
[56] S. Abed, M. Alshayeji, and S. Sultan. 2019. Diacritics effect on Arabic speech recognition. *Arab. J. Sci. Eng.* 44, 11 (2019), 9043–9056.
[57] K. Darwish, H. Mubarak, and A. Abdelali. 2017. Arabic diacritization: Stats, rules, and hacks. In *3rd Arabic Natural Language Processing Workshop*. 9–17.
[58] R. Abdelmalek and Z. Mnasri. 2016. High quality Arabic text-to-speech synthesis using unit selection. In *13th International Multi-conference on Systems, Signals & Devices (SSD'16)*. IEEE, 1–5.
[59] A. Alsaif, N. Albadrani, A. Alamro, and R. Alsaif. 2017. Towards intelligent Arabic text-to-speech application for disabled people. In *International Conference on Informatics, Health & Technology (ICIHT'17)*. IEEE, 1–6.
[60] O. Abdo, S. M. Abdou, and M. Fashal. 2017. Building audio-visual phonetically annotated Arabic corpus for expressive text to speech. In *INTERSPEECH*, 3767–3771.
[61] I. H. Ali, Z. Mnasri, and Z. Laachri. 2019. Gemination prediction using DNN for Arabic text-to-speech synthesis. In *16th International Multi-conference on Systems, Signals & Devices (SSD'19)*. IEEE, 366–370.
[62] Z. Oumaima, M. Abdelouafi, and M. El Hadi. 2018. Text-to-speech technology for Arabic language learners. In *IEEE 5th International Congress on Information Science and Technology (CiSt'18)*. IEEE, 366–370.
[63] I. Rebai and Y. BenAyed. 2016. Arabic speech synthesis and diacritic recognition. *Int. J. Speech Technol.* 19, 3 (2016), 485–494.
[64] F. Fahmy, M. Khalil, and H. Abbas. 2020. A transfer learning end-to-end arabictext-to-speech (TTS) deep architecture. *arXiv preprint arXiv:2007.11541* (2020).
[65] H. A. Elharati, M. Alshaari, and V. Z. Këpuska. 2020. Arabic speech recognition system based on MFCC and HMMs. *J. Comput. Commun.* 8, 03 (2020) 28.

[66] H. Bouressace and J. Csirik. 2019. A convolutional neural network for Arabic document analysis. In *IEEE International Symposium on Signal Processing and Information Technology (ISSPIT'19)*. IEEE, 1–6.
[67] M. Eltay, A. Zidouri, and I. Ahmad. 2020. Exploring deep learning approaches to recognize handwritten Arabic texts. *IEEE Access* 8 (2020), 89882–89898.
[68] A. Arora, C. C. Chang, B. Rekabdar, B. BabaAli, D. Povey, D. Etter, D. Raj, H. Hadian, J. Trmal, P. Garcia, and S. Watanabe. 2019. Using ASR methods for OCR. In *International Conference on Document Analysis and Recognition (ICDAR'19)*. IEEE, 663–668.
[69] H. Mohamad, S. A. Hashim, and A. H. Al-Saleh. 2019. Recognize printed Arabic letter using new geometrical features. *Indon. J. Electr. Eng. Comput. Sci.* 14, 3 (2019), 1518–1524.
[70] K. Mohammad, A. Qaroush, M. Ayesh, M. Washha, A. Alsadeh, and S. Agaian. 2019. Contour-based character segmentation for printed Arabic text with diacritics. *J. Electron. Imag.* 28, 4 (2019), 043030.
[71] M. E. Mustafa and M. K. Elbashir. 2020. A deep learning approach for handwritten Arabic names recognition. *Int. J. Adv. Comput. Sci. Applic.* 11, 1 (2020).
[72] A. Qaroush, B. Jaber, K. Mohammad, M. Washaha, E. Maali, and N. Nayef. 2019. An efficient, font independent word and character segmentation algorithm for printed Arabic text. *J. King Saud Univ.-Comput. Inf. Sci.* DOI: https://doi.org/10.1016/j.jksuci.2019.08.013
[73] I. S. Al-Sheikh, M. Mohd, and L. Warlina. 2020. A review of arabic text recognition dataset. *Asia-Pac. J. Inf. Technol. Multimedia* 9, 1 (2020), 69–81.
[74] T. Milo and A. G. Martínez. 2019. A new strategy for Arabic OCR: Archigraphemes, letter blocks, script grammar, and shape synthesis. In *3rd International Conference on Digital Access to Textual Cultural Heritage (DATeCH'19)*. Association for Computing Machinery, New York, NY, 93–96. 2019. DOI: https://doi.org/10.1145/3322905.3322928
[75] S. M. Darwish and K. O. Elzoghaly. 2020. An enhanced offline printed Arabic OCR model based on bio-inspired fuzzy classifier. *IEEE Access* 8 (2020), 117770–117781.
[76] M. Kadi and M. Nasri. 2019. Isolated Arabic characters recognition using a robust method against noise and scaling based on the ńhough transformż. *Int. J. Inf. Sci. Technol.* 3, 4 (2019), 34–43.
[77] W. N. Hussein and H. N. Hussain. 2019. A design of a hybrid algorithm for optical character recognition of online hand-written Arabic alphabets. *Iraqi J. Sci.* 60, 9 (2019), 2067–2079.
[78] M. W. Ok and K. Rao. 2017. Using a digital pen to support secondary students with learning disabilities. *Interv. School Clin.* 53, 1 (2017), 36–43.
[79] Wizcomtech. 2020. The freedom to read. Retrieved from https://www.wizcomtech.com.
[80] C-Pen. 2020. The original pen scanner brand. Retrieved from https://cpen.com/.
[81] IRISPen. 2020. The digital highlighter that types what you scan! Retrieved from https://www.irislink.com/EN-JO/c1708/IRISPen-Air-7—Portable-Digital-Highlighter.aspx.
[82] WorldPenScan X. Entry & Translation Retrieved on December 2020 from http://www.penpowerinc.com/product.asp?sn=735.
[83] Livescribe. A pen for every occasion. Retrieved on December 2020 from https://us.livescribe.com/collections/smartpens.
[84] K. C. Huang, C. K. Sun, D. Y. Huang, Y. C. Chen, R. C. Chang, S. W. Hsu, C. Y. Yang, and B. Y. Chen. 2020. Glissade: Generating balance shifting feedback to facilitate auxiliary digital pen input. In *CHI Conference on Human Factors in Computing Systems*, 1–13.
[85] C. M. Chen, J. Y. Wang, and M. Lin. 2019. Enhancement of English learning performance by using an attention-based diagnosing and review mechanism in paper-based learning context with digital pen support. *Univ. Access Inf. Soc.* 18, 1 (2019), 141–153.
[86] C. M. Chen, C. C. Tan, and B. J. Lo. 2016. Facilitating English-language learners' oral reading fluency with digital pen technology. *Interact. Learn. Environ.* 24, 1 (2016), 96–118.
[87] C. C. Tan, C. M. Chen, and H. M. Lee. 2020. Effectiveness of a digital pen-based learning system with a reward mechanism to improve learners' metacognitive strategies in listening. *Comput. Assist. Lang. Learning.* 33, 7 (2020), 1–26.
[88] N. Choi, S. Kang, and J. Sheo. 2020. Children's interest in learning English through picture books in an EFL context: The effects of parent–child interaction and digital pen use. *Educ. Sci.* 10, 2 (2020), 40.
[89] P. Krish. 2020. The use of the audio pen in enhancing reading skills among preschool children. *Int. J. Inf. Educ. Technol.* 10, 5 (2020).

Received March 2020; revised August 2020; accepted September 2020